Name: _____
Surname: _____
Date: _____

Multiple Choice (10 points each)

Instructions

Circle the correct answer for each question, as each question has **only one** correct answer.
Provide reasoning for your answers in the space next to each question.
A correct answer always awards full points (+10 pts).
An incorrect answer with sensible reasoning awards half points (+5 pts).
An incorrect answer with faulty or no reasoning subtracts points (-3 pts).
No answer gives zero points (neither adds nor subtracts points).
Assume each code cell is **run in isolation**, meaning no other code has been run before it.

Questions

1. After running this code, what will be the final value of the **variable "a"**?

```
1 a = 4 - 4 / 0
2 a = abs(a)
```

   (a) `a = ∞`
   (b) `a = 0`
   (c) The code will raise an error.

2. Is there any difference between the commands in lines 2 and 3?

```
1 x = 3.14
2 y = str(x)  # Line 2
3 z = f"{x}"  # Line 3
```

   (a) Both commands produce the same output (meaning that `y = z`).
   (b) The commands produce a different output (meaning that `y ≠ z`).
   (c) One of them raises an error.

3. Which of the following lines of code will **raise an error**?

```
1 sports = ("football", "chess", "dancing")
2 sports[1] = "basketball"        # Line 2
3 print(f"I love {sports[-1]}")   # Line 3
4 print(sports[0][4])             # Line 4
```

   (a) Line 2.
   (b) Line 3.
   (c) Line 4.

4. Which message will **never be printed**, no matter the value of "name"?

```
name = "Daniel"  # Anything goes here

if name[0] == "d":
    print("Your name starts with d")
elif name[0].lower() == "d":
    print("Your name starts with D")
elif (type(name) == int) or (type(name) == float):
    print("A number is not a string!")
else:
    print("Your name starts with a letter other than d")
```

(a) "Your name starts with D"

(b) "A number is not a string!"

(c) "Your name starts with a letter other than d"

5. After running this code, what will be the final value of the **variable "count_up"**?

```
count_up = 1

while count_up <= 4:
    count_up += count_up
```

(a) `count_up = 4`

(b) `count_up = 8`

(c) The code will raise an error / never stops.

6. After running this code, what will be the length of "ls_end"?

```
def fibonacci(n: int, ls: list=[1, 1]) -> list:
    # Add 'n' more values to the list using Fibonacci logic
    # where x_(i) = x_(i-2) + x_(i-1)
    for _ in range(n):
        val = ls[-2] + ls[-1]
        ls.append(val)
    return ls

ls_test1 = fibonacci(3)
ls_test2 = fibonacci(3, ls=[1, 1, 2])
ls_end = fibonacci(3)
```

(a) `len(ls_end) <= 5`

(b) `len(ls_end) > 5`

(c) The code will raise an error.

Answers: 1-c, 2-a, 3-a, 4-b, 5-b, 6-b

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

7. Safe Division (20 points)

Write a function named `safe_divide(numerator, denominator)`. This function must be able to handle specific edge cases and return values in specific formats as described below:

- If `denominator` is 0, the function should **print** the warning message `"Warning: Division by zero!"` and **return** 10^{10}. *(5 points)*
- If the result of the division is an integer (e.g., 10 divided by 5), the function should return an `int` value. *(10 points)*
- For all other cases, the function should return a `float` value representing the result of the division. *(5 points)*

Answer:

The function receives two numbers, `numerator` and `denominator`. Because the output depends on the case, we will use an if-elif-else structure with three branches, one for each case.

Division by zero. The first branch must be the check `if denominator == 0`. We have to check it before dividing, because otherwise Python would raise a `ZeroDivisionError`. Inside it, we use `print("Warning: Division by zero!")`, and we return 10^{10}. Be careful: in Python the power is `10**10`, while `10^10` is a different operation (bitwise XOR) that returns 0. *(+5 points)*

Integer results. For the second case, we need to know if the division has no remainder. We can check it with the modulo operator, `numerator % denominator == 0`. Alternatively, we can compute the result first, and compare it against its integer version, `result == int(result)`. If it is true, we return an integer: `return int(numerator // denominator)`. Simply returning

`numerator / denominator` is not enough, because the operator `/` always returns a float (`10 / 5` is `2.0`). *(+10 points)*

All other cases. We only get to the last branch (`else`) if the denominator is not 0 and the division is not exact, so we return `numerator / denominator`, which is a float. *(+5 points)*

(Optional, code)

```
def safe_divide(numerator, denominator):
    # Case 1: division by zero
    if denominator == 0:
        print("Warning: Division by zero!")
        return 10**10
    # Case 2: the division is exact, so we return an integer
    elif numerator % denominator == 0:
        return int(numerator // denominator)
    # Case 3: any other case, we return a float
    else:
        return numerator / denominator
```

8. Planning App (20 points)

You have a dictionary that lists people and their hobbies, as shown below:

```
dict_people = {
    "Amy": ["cinema", "dancing", "health", "history", "travel"],
    "Martha": ["health", "running", "travel"],
    "Rory": ["cinema", "dancing", "history", "running"]
}
```

Write a function named `match_people(dict_people)`. This function should take a dictionary (like the one above) as input and print all pairs of people who share at least one hobby. *(12 points)* For example, given the dictionary above, the output should be:

```
"Amy and Martha: health, travel"
"Amy and Rory: cinema, dancing, history"
"Martha and Rory: running"
```

Ensure the function handles errors gracefully. If the input is not a dictionary with lists of strings, the function should stop and print the message "Input is not the expected format". *(4 points)*

Avoid redundancy in the output. For instance, if "Amy and Martha" is printed, do not print "Martha and Amy". *(4 points)*

Answer:

The function receives the dictionary, and it **prints** the results (no `return` is required).

Input validation. Before anything else, we check the format of the input. If `type(dict_people) != dict`, we print `"Input is not the expected format"` and stop the function with `return`. Then we loop through the values of the dictionary, `for hobbies in dict_people.values()`, and check that each one is a list. In a nested loop, we check that each hobby is a string. In any other case, we print the same message and `return`. Using `return` stops the function, so the rest of the code is never run with a wrong input. *(+4 points)*

Pairs without redundancy. We need to compare every person against every other person, but each pair only once. First, we get the names as a list, `names = list(dict_people.keys())`. Then we use a nested loop: the outer loop is `for i in range(len(names))`, and the inner loop is `for j in range(i + 1, len(names))`. Starting the inner loop at `i + 1` makes sure

that we never compare a person with themselves, and that we never repeat a pair (we get "Amy and Martha" but not "Martha and Amy"). *(+4 points for avoiding the redundancy, +4 points for the loops)*

Shared hobbies. Inside the inner loop, we convert the hobby lists of both people into sets, and use the intersection to find the hobbies they share. We then use an if-clause to check that the set is not empty, `len(shared) > 0`. If so, we print the message using an f-string. As the hobbies are in a set, we sort them first, and use the method `join` to separate them with commas: `", ".join(sorted(shared))`. *(+4 points)*

(Optional, code)

```
def match_people(dict_people: dict) -> None:
    # Check that the input has the expected format
    if type(dict_people) != dict:
        print("Input is not the expected format")
        return
    for hobbies in dict_people.values():
        if type(hobbies) != list:
            print("Input is not the expected format")
            return
        for hobby in hobbies:
            if type(hobby) != str:
                print("Input is not the expected format")
                return

    names = list(dict_people.keys())
    # Compare every pair of people once
    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            name_a = names[i]
            name_b = names[j]
            # Hobbies in common
            shared = set(dict_people[name_a]).intersection(set(dict_people[
name_b]))
            # Print only if there is at least one
            if len(shared) > 0:
                print(f"{name_a} and {name_b}: {', '.join(sorted(shared))}")
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.